

In-Class Exercise Sheet 9

In this lecture, we look at large language models (LLMs) through the lens of reproducibility: trade-offs when packaging an experiment that calls an LLM, how to make the generated output as repeatable as possible, and limitations w.r.t. reproducibility.

1. You are using an LLM in your experiment. For the reproduction package you can either (i) run an open-weight model *locally*, inside the container, or (ii) call a *remote* LLM via an API. Compare the two architectures w.r.t. reproducibility by filling in the table.

	Local LLM inside container	Remote API
Self-contained?	_____	_____
Container/artifact size	_____	_____
Local hardware requirements	_____	_____
Long-term availability	_____	_____
Cost to re-run	_____	_____
Transparency	_____	_____

2. If you use a remote LLM in your experiments, what can you do to ensure long-term reproducibility?

3. When using a remote LLM, an API key is used for authentication and billing. It should hence be kept secret. How can you securely provide an API key to the container?

4. Fill in the blanks in the following text about improving reproducibility of LLM outputs.

Where the API supports it, ...

... set the sampling _____ to 0 to obtain the most (near-)deterministic results; higher values increase randomness.

... pass a fixed _____, to pin the pseudo-random choices used during sampling, so that identical requests are sampled the same way.

Note: With reasoning models like GPT 5.5, these parameters are usually not supported.

- (a) Why is exact (bitwise) reproducibility *not* guaranteed, even when applying the settings above?

(b) What should therefore go into the reproduction package?

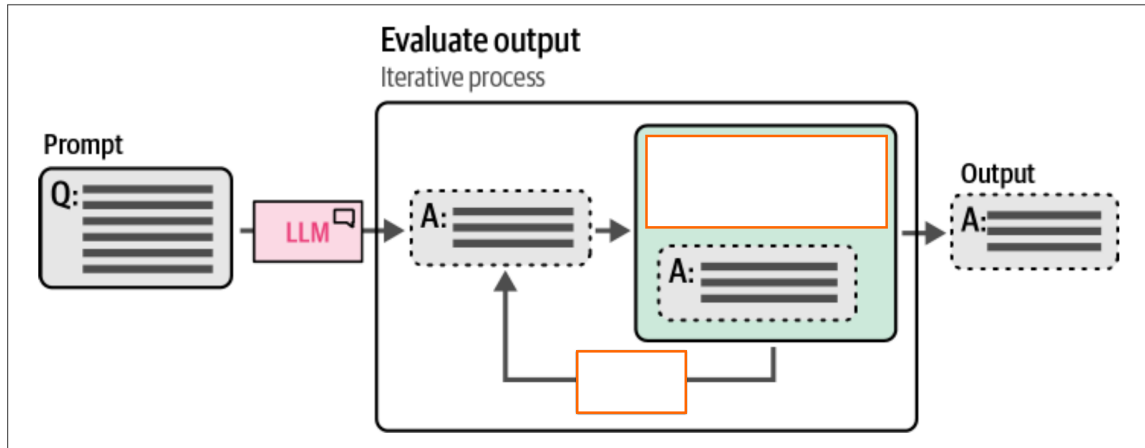
5. A prompt can be decomposed into several components, each playing a different role. For each example segment below, name the role the component plays.

Example segment in the prompt	Component
“You are an expert research software engineer.”	_____
“Extract the software, datasets, and hardware used in the paper below.”	_____
“Focus on the points needed to reproduce the experiment.”	_____
“Answer with a JSON object, using one property per item.”	_____
“The output is for reviewers assessing reproducibility.”	_____
“Keep the JSON values concise and factual.”	_____
“Paper: <text of the paper . . . >”	_____

6. There are different approaches to ensure that an LLM follows a certain output structure. Match a technique name to each description.

Technique Name	Description
_____	Show the model correctly formatted outputs so it imitates the format. Simple, but <i>no guarantee</i> .
_____	Supply the output skeleton yourself (e.g. the JSON keys and braces) and let the model generate <i>only</i> the values it does not know beforehand. The scaffolding cannot be malformed because the model never emits it.
_____	Restrict generation to tokens allowed by a grammar/schema, so the output is valid <i>by construction</i> .
_____	Validate the output (e.g. against a JSON Schema); on failure, re-prompt or ask another tool/model to repair it.

7. Fill in the orange boxes in the figure below showing an evaluation-retry-loop for LLM-based JSON generation.



Source: “Hands-On Large Language Models” by J. Alammam and M. Grootendorst (O’Reilly, 2024).

8. You want the model to return a JSON instance that is valid for a given schema. Compare the following approaches:
- (A) Pasting the schema into the prompt and asking the LLM to produce a JSON instance that is valid for it.
 - (B) Using proper constrained decoding (structured outputs).

9. Constrained decoding tools accept only a *subset* of JSON Schema. Fill in the table, comparing the support of some JSON schema features in structured outputs (constrained decoding) across OpenAI, Anthropic and llama.cpp.

Feature	OpenAI	Anthropic	llama.cpp
additional-Properties	_____	_____	_____
Optional properties	_____	_____	_____
minimum, maximum	_____	_____	_____
pattern, format	_____	_____	_____
anyOf	_____	_____	_____
oneOf	_____	_____	_____

10. A model call uses constrained decoding with JSON Schema and succeeds. Which statements are true? Check all that apply.
- ☐ The output is syntactically correct JSON.
 - ☐ The output conforms to the declared types and required properties of the schema.
 - ☐ The numbers and strings in the output are guaranteed to be factually correct.
 - ☐ The output is guaranteed to be free of hallucinated content.
 - ☐ Semantic constraints not expressible in the schema (e.g. “end_date after start_date”) must still be checked separately.
 - ☐ A schema may be rejected or silently altered, because each tool supports only a subset of the JSON Schema specification.

11. Fill in the blanks in the following Python script that performs text generation via the OpenAI API.

```
1 import os
2 from openai import OpenAI
3
4 # Never hard-code the key: read it from the environment.
5
6 client = OpenAI(api_key=_____) # (1)
7
8 response = client.chat.completions.create(
9     model = "gpt-5.5",
10    # Turn off reasoning to enable temperature parameter
11    reasoning_effort = "none",
12    messages = [
13        {"role": "developer",
14         "content": "You are a helpful assistant."},
15        {"role": "user",
16         "content": "Define reproducibility in one sentence."},
17    ],
18    _____=0.0, # (2) most deterministic decoding
19
20    _____=42, # (3) reproducible sampling
21 )
22
23
24 print(response._____) # (4) the generated text
```


12. Fill in the blanks in the following Python script that performs text generation with structured outputs (constrained decoding) via the OpenAI *Responses* API.

```
1  ... # Imports, initialize OpenAI API client
2  schema = {
3      "type": "object",
4      "properties": {
5          "value": {"type": "number"},
6          "unit": {"type": "string"},
7      },
8      "required": [_____], # (1)
9
10     "additionalProperties": _____, # (2)
11 }
12 response = client.responses.create(
13     model = "gpt-5.5",
14     reasoning = {"effort": "low"},
15     input = [{"role": "user",
16               "content": "How hot is boiling water at sea level?"}],
17     text = {
18         "format": {
19             "type": "json_schema",
20             "name": "measurement",
21             "schema": schema,
22
23             "strict": _____, # (3)
24         }
25     },
26 )
```